

A Quorum-Based Total Order Broadcast Protocol for Distributed Intelligence Databases

Dalla Croce Enrico, Dematté Luca

Distributed Systems 1 Project – July 24, 2026

1 Project Structure

1.1 Subpackage

We created a subpackage named `cs` located at `it.unitn.ds.cs`. The code related to our abstractions is located under the `cs` package.

1.2 Naming Convention

To easily identify our classes all class names have the prefix `CS`.

2 Design

To keep the explanation concise, we focus solely on the aspects that complement the project instructions: our assumptions, additional features, and custom solutions.

2.1 Assumptions

2.1.1 Project Instructions

- Communication channels are assumed to be reliable and FIFO
- A strict majority of replicas never crashes.
- Replicas fail by crashing and do not recover.
- Static system membership: no new replicas may join after initialization.

2.1.2 UUID uniqueness

We assume that the generated UUIDs are unique. Replicas independently generate UUIDs and so technically it can happen that two independent UUIDs match, making them not unique.

It has to be noted that we rely on the `UUID` class included in the Java SDK which offers some guarantees.

The documentation for the method `public static UUID randomUUID()` states: "The UUID is generated using a cryptographically strong pseudo random number generator."

We can also look at the problem from a statistical point of view. A UUID represents a 128bit value, the chance of two random UUID being identical is $\frac{1}{2^{128}} = 3 \times 10^{-39}$, which is negligible

2.1.3 Replica Crash

Because channels are reliable if a Replica does not Ack and Update message it can be considered crashed

2.1.4 Timeout values

The timeouts values are all based on `getMaxLatencyPlusTolerance()` which internally takes in consideration the network latency and the number of active replicas. Through trial and error we found values that made the system responsive and avoided false positives. We believe that the values do would not need to be recomputed with the system scaling of the number of replicas changing.

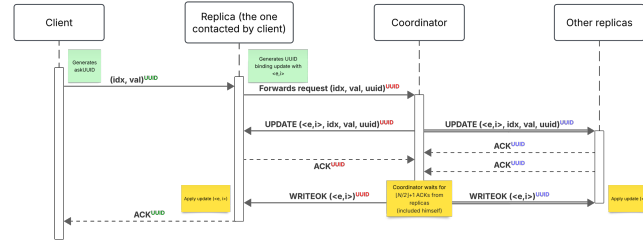


Figure 1: Update Protocol Diagram

2.2 AskSystem

To handle timeouts we created a wrapper around `tell()` that emulates the behavior of `ask()`.

We initially attempted to follow Akka Ask pattern using the provided `ask()` method, which returns a `CompletionStage(future)` that completes when a response is received. However, the future is completed on whatever thread delivers the response which is not guaranteed to be the requesting sending actor thread. Mutating actor state inside the callback introduce race conditions. Akka ask documentation: *"do not call methods or access mutable state [...] from within the callback. This [...] may introduce synchronization bugs and race conditions. [...] there is not yet a way to detect these illegal accesses at compile time."*

The standard solution is to use `pipeToSelf`, which delivers the result as a message to the actor mailbox, where it is processed like any other message. But the project requires to use the `tell()` method that emulates network latency. And this cannot be done using provided methods.

Therefore we designed our own wrapper called `askSystem` built on top of the custom `tell()` to emulate the ask pattern which makes very easy handling timeouts. `AskSystem` enables an actor to send messages and register a callback to be executed when a response is received and/or when the specified timeout expires. All messages sent through `AskSystem` must extend `CAskMessage` which has an UUID as an attribute used to correlate a request with a response and trigger the correct callback.

When a message is sent a `CAskTimeout` to self is scheduled through a timer. If the response is received in time the timer is canceled and the callback fired, otherwise if the `CAskTimeout` message is received the callback is still fired but with the information that the timer expired.

Example: Figure 1 where the UUIDs are shown and color coded for each message

2.3 Update Protocol

The `ForwardRequest` and the `Update` message share a UUID which is used to identify the `WriteRequest` and the corresponding `Update` which can be confused if two replicas ask to update the same value at the same time.

Example:

- ReplicaA and replicaB send `WriteRequest` (index=2, value=7)
- Coordinator sends `Update` (<0, 1>, index=2, value=7)
- Coordinator sends `Update` (<0, 2>, index=2, value=7)

Which one is the answer to which request? The UUID takes care of matching the Request and the Update

2.3.1 Crash Detection

During the update protocol the coordinator can detect crashed replicas that do not answer with an Ack.

The coordinator after detecting a crash removes the replica from its list of active replicas and sends a `CSCrashNotice` message to the previous replica so that it can update its own list of active replicas and the pointer to the next replica.

Originally crashed replicas are discovered only during the Election process when passing the message through the ring. This extra measure speeds up the Election process avoiding some unnecessary timeouts as crashed replicas can be discovered before the Election.

2.4 Updates Logging

Update logging is taken care by the class CSUpdateLogger. Every replica has an attribute updateLogger which is used to:

- Log write requests coming from clients
- Log updates coming from the coordinator

It is important to log also requests as during the coordinator recovery (subsection 2.6.3) we need to process the requests that have not been fulfilled and not only the updates that did not complete.

2.5 Coordinator

The coordinator acts differently from replicas in almost every operation. We decided to introduce custom behavior than making it behave similarly to replicas as this would have required sending messages to itself.

As a result all local operations are performed locally skipping the sending of any message.

2.5.1 Heartbeat

The heartbeat detection is not done through the AskSystem which works only during transactions. We instead opted for a timer based system that in contrast works also during idle periods.

When a replica becomes a coordinator a recurring self message is scheduled, which upon receiving triggers the broadcast of the heartbeat message to all replicas. Each replica schedules a recurring self message, which triggers a check that the heartbeat has been received during the interval.

A flag is raised when a heartbeat is received and lowered after every check. If the flag is not found raised during a check, an election is initiated.

The replica start checking for a heartbeat with some delay to allow the coordinator to spin up. In addition the checking interval is a little longer than the beat interval to allow for slowdowns in the communication and processing. Everything combined absorbs propagation delay and avoids false positives.

2.6 Election

The ring structure needed during the election is maintained through a series of pointers. Each replica has an attribute next which is a reference to the next Actor in the ring

2.6.1 Crash detection

When forwarding the election message through the ring if the timeout expires the replica knows that its next has crashed so it can remove it from its list of active replicas and update its next

2.6.2 Non Blocking

To make sure that the election process terminates we do two things:

- If a crash is detected the replica tries to contact the next replica that should be alive and continues indefinitely until the election can continue
- When passing the election message with the chosen coordinator if the previous replica in the ring detects the crash of the winner it removes it from the election message, computes the new winner and continues to circulate the message

2.6.3 Recovery

After sending the synchronization message the coordinator start the recovery of incomplete updates.

Step 1: compute oldest update completed across all replicas

Step 2: send incomplete updates

```
foreach update ∈ incompleteUpdates do
  | Run the update protocol for update
end
```

Step 3: handle write request received during the election

```
foreach writeRequest ∈ writeRequests do
  | update ← create update with a key in the new epoch for writeRequest
  | Run the update protocol for update
end
```

3 Implementation

3.1 Custom ask implementation

The core of the AskSystem is the ask() function that emulates the Akka ask pattern. The callback is defined with a functional interface (Figure 3) used to mimic ask original behavior (Figure 2).

Figure 5 shows an example of ask() in use. the return value type is parametric and so casting is automatically performed, res will be of the type specified.

```
CompletionStage result = AskPattern.ask(actor, message, timeout);
Figure
result.whenComplete(
    (reply, failure) -> {
        if (...) {...}
        else {...}
    });
```

Figure 2: ask in Akka Classic

```
@FunctionalInterface
public interface Callback<T extends CSAskMessage> {
    void handle(T response, boolean timedOut);
}
```

Figure 3: ask callback parameter type

```
public <T extends CSAskMessage> void ask(
    CSAskMessage msg, ActorRef destination,
    Duration timeout, Callback<T> callback
)
```

Figure 4: ask firm

```
askSystem.<CSUpdate>ask(new CSWriteForward(...),
    replicas.get(this.coordinatorId),
    timeout,
    (res, timedOut) -> {
        if (!timedOut) {...}
        else {...}
    })
```

Figure 5: ask usage example

3.1.1 WriteOk Timeout

When a replica sends its Ack through ask a timer starts. To avoid the timer from expiring we need to send a WriteOk containing the correct askUUID to the replica. The problem is that the coordinator must broadcast the WriteOk message when the quorum is reached.

To comply with AskSystem the coordinator would need to:

Step 1: Collect Acks

```
while collecting Acks to reach quorum do
  | Track askUUID associated with received Ack
end
```

Step 2: Send WriteOk messages

```
foreach replica ∈ replicas do
  if replica participated in reaching the quorum then
    | Send WriteOk with correct askUUID to replica
  end
  else
    | Send WriteOk with random askUUID to replica
  end
end
```

Step 3: Answer to successive Acks for the update

```
foreach ack ∈ successiveAcks do
  | Send WriteOk with askUUID learned from ack
end
```

This would work but it would not be very practical and it would require the coordinator to send more messages than the protocol specifies.

Our solution instead is for each replica to keep a map of the update key and the askUUID used to send the Ack. In HandleUpdate() the mapping is recorded in Map<CSUpdateKey, UUID> updatesAskUUIDs. Then when the WriteOk message is received it is not feed directly to askSystem.handleResponse(), previously a new WriteOk message is created with the correct askUUID retrieved from the map.

```
public final void handleWriteOk(CSWriteOk msg) {
    UUID askUUID = this.updatesAskUUIDs.get(msg.getKey());
    this.askSystem.handleResponse(new CSWriteOk(msg.getKey(), askUUID));
}
```

3.2 Update Logging

We leverage two different HashMaps:

- Map<UUID, CSUpdateData> updates
- Map<CSUpdateKey, UUID> keyToUUIDBindings

When a replica receives a WriteRequest from the client the coordinator has not assigned a key yet. The only identifying information is the generated UUID, so the update is first inserted into updates.

When replica receives an Update message it binds the key to the UUID through keyToUUIDBindings.

Upon receiving the WriteOk which identifies the update only by its key we need to gather the update data chaining the two HashMaps: updates.get(keyToUUIDBindings.get(key)). The data contains information on the write to apply(index and value) and the reference to the client that is waiting for an Ack to the WriteRequest.

4 Information about the use of LLM tools

- Gemini: questions about Java.
- ChatGPT: (before starting with the development) questions about the codebase to understand better how to use it.
- OpenCode (Big Pickle): reasoning on the codebase and providing pseudo code.
- Claude, Mistral: help in writing the report. Used to write drafts and make user that all arguments were covered correctly.